SajdaTime — Requirements and Architecture

Version 1.1.1 · Android and Wear OS · Kotlin + Jetpack Compose

This document is the handoff record: what was built, why each decision was made, the exact business rules an iOS port must reproduce, and what deliberately was not built.

---

1. Product intent

SajdaTime is a charity project (sadaqah jariyah). It is free forever, carries no ads and no in-app purchases, and collects nothing.

Three constraints shaped every technical decision:

Constraint	Consequence
Zero running cost	Prayer times are computed on-device. There is no server, and no paid API.
Privacy by design	Only coarse location, only in the foreground, never transmitted, never backed up.
Solo maintainer	Minimal dependencies, no build-time codegen, no framework the author must relearn.

---

2. Technology choices

Area	Choice	Why
Language	Kotlin	Official, concise.
UI	Jetpack Compose + Material 3	Less code than XML, better animation, one source of truth for theming.
Min SDK	24 (Android 7.0)	Covers the overwhelming majority of active devices worldwide without carrying Android 5 baggage.
Target SDK	36	Meets Google Play's requirement. Compiled against SDK 37.
Calculation	`com.batoulapps.adhan:adhan:1.2.1` (MIT)	Fully offline, well tested, zero transitive dependencies.
Storage	DataStore Preferences	Async, no SharedPreferences main-thread traps.
Scheduling	`AlarmManager` + `WorkManager`	Exact alerts plus a daily safety net.
PDF	Android `PdfDocument`	Built into the platform. No library, no licence, no APK weight.
Qibla	Own great-circle maths + `GeomagneticField`	The platform ships the World Magnetic Model already. No library needed.
Watch	Wear Compose + ProtoLayout tiles	Standalone watch app sharing the same calculation module.
Networking	`HttpURLConnection` + `org.json`	One optional GET request does not justify Retrofit/OkHttp/Moshi.
Java 8+ APIs	Core library desugaring	Gives `java.time`, including the Hijri calendar, down to API 24.

Deliberately not used

- **Google Play Services location.** The platform `LocationManager` is enough for coarse accuracy, removes a proprietary dependency, and keeps the app working on de-Googled devices. Prayer times shift by well under a minute across a coarse-location cell.
- **Material You dynamic colour.** A wallpaper-derived palette would break the verified contrast guarantees and the deliberate green/gold identity.
- **Dependency injection framework.** Three repositories constructed in one ViewModel.
- **Navigation library.** Two screens and an onboarding flow. A boolean is enough.

2a. Module layout

```
:core   Prayer and Qibla calculation. No android.* imports. Shared by phone and watch.
:app    The phone app.
:wear   The Wear OS app and tile.
```

`:core` is an Android library only so that AGP's built-in Kotlin and core library desugaring apply. The code inside is plain Kotlin and lifts straight into a Kotlin Multiplatform or iOS target.

---

3. Prayer calculation — the business rules

This section is the specification an iOS port must match exactly.

3.1 Method selection

`CalcMethod.AUTO` resolves at calculation time from the user's sect:

- Sunni → Muslim World League
- Shia → Jafari (Ithna Ashari)

A user who never opens advanced settings still gets a correct convention. An explicitly chosen method always overrides AUTO.

Which methods a school is offered is one definition, `CalcMethod.offeredTo(sect)`: Jafari and Tehran are offered to Shia only (they carry the post-sunset Maghrib of §3.4), every other named method to Sunni only, and AUTO to both. Two things depend on it and a port must keep them together: the list the user picks from is filtered by it, and **changing school discards a stored method the new school was never offered, falling back to AUTO** — in both directions, on both the phone and the watch. Otherwise a Sunni convention survives a switch to Shia and the Jafari Maghrib is silently lost, or the reverse.

The app never chooses a method for the user by any proxy — not madhab, latitude, country or a preference question. It asks once at setup, keeps a permanent "Different from your mosque?" explainer on the Times screen, and shows a dismissible prompt above 45° of latitude while the method is still AUTO. Reasoning and measurements: `HANDOVER.md` §5.17 and §10.

3.2 Asr (madhab)

Asr depends only on the shadow ratio:

Madhab	Shadow ratio	adhan value
Hanafi	2× object length	`Madhab.HANAFI`
Shafi'i, Maliki, Hanbali	1× object length	`Madhab.SHAFI`
Jafari	1× object length	`Madhab.SHAFI`

Three of the four Sunni madhabs therefore produce identical times. This is correct, not a bug, and is asserted in `DeterminismTest`.

3.3 Angles

Method	Fajr	Maghrib	Isha
Muslim World League	18°	sunset	17°
Umm al-Qura	18.5°	sunset	sunset + 90 min (120 in Ramadan)
Jafari (Ithna Ashari)	16°	**4° below horizon**	14°
Tehran (Geophysics)	17.7°	**4.5° below horizon**	14°

Other Sunni conventions use adhan's bundled parameters unchanged.

3.4 The Jafari Maghrib rule — the one non-obvious piece

For Sunni conventions, Maghrib is sunset. For Jafari and Tehran, Maghrib is when the redness leaves the eastern sky, defined as the sun sitting 4° (4.5° for Tehran) below the horizon. That is typically **15–20 minutes after sunset**, and it is the moment fasts are broken — so getting it wrong is not cosmetic.

adhan exposes no Maghrib angle. Neither `adhan-java` nor `adhan2` models one.

Rather than hand-rolling solar astronomy, the engine reuses adhan's own validated maths: adhan's Isha calculation is exactly "the evening time at N degrees of solar depression". So the engine runs a second calculation with `ishaAngle` set to the Maghrib angle and reads the resulting Isha.

```
val probe = CalculationParameters(0.0, 4.0, CalculationMethod.OTHER)
val jafariMaghrib = PrayerTimes(coords, date, probe).isha
```

This was verified to the minute against the Aladhan API's independent Shia Ithna-Ashari implementation — all six times matched exactly for Tehran on 21 June 2026, including Maghrib at 19:42 against a sunset of 19:24. See `PrayerEngineTest`.

An iOS port using adhan-swift must reproduce this, since adhan-swift has the same gap.

3.5 High latitude rule

Above roughly 48° latitude the sun never dips far enough below the horizon in summer for a true Fajr or Isha to exist. A fallback is mandatory.

The engine uses `HighLatitudeRule.TWILIGHT_ANGLE`, which divides the night in proportion to each prayer's twilight angle.

adhan's own default, `MIDDLE_OF_THE_NIGHT`, is unusable at UK latitudes: for London on 21 June it collapses Fajr and Isha onto the *same instant* (both 01:02), leaving no window for Isha at all. `TWILIGHT_ANGLE` produces Fajr 02:31 and Isha 23:27 for the same date, matching Aladhan exactly.

This is set explicitly in `PrayerEngine`, not left to the library default.

3.6 Ramadan (Umm al-Qura only)

The official Umm al-Qura calendar stretches the Isha interval from 90 to 120 minutes during Ramadan. adhan implements only the 90-minute rule, so the engine adds the extra 30 minutes itself.

Ramadan is detected as Hijri month 9 via `java.time.chrono.HijrahDate` — and `HijrahChronology` *is* the Umm al-Qura calendar, so the two agree by construction. The adjustment applies to no other method. Since 1.4.0 that call lives in one file, `core/HijriCalendar.kt` (3.10).

3.7 Determinism

adhan rounds its results to the nearest minute but leaves the millisecond field carrying the wall-clock time of the call. Two identical computations therefore return instants a few milliseconds apart.

The engine truncates every result to the minute. Without this, the countdown would jitter and alarms would land at an arbitrary point within their minute. Guarded by `DeterminismTest`.

3.8 Next prayer

`nextPrayer` scans **yesterday, today and tomorrow**, not just today. At high latitudes Isha can fall after midnight and therefore belongs to the previous calendar day's timetable while still being genuinely upcoming. Scanning today alone would silently skip it. The alarm scheduler applies the same rule.

Sunrise is displayed but is never returned as "the next prayer" and is never notified.

3.9 Qibla

The Qibla is the initial great-circle bearing from the user to the Kaaba (21.4224779 N, 39.8251832 E), measured clockwise from **true** north.

Two things are easy to get wrong here, and both are handled:

1. **Great circle, not flat map.** Treating latitude and longitude as flat x/y puts London at about 127 degrees instead of the correct 119. The error grows with latitude and distance.
2. **True north, not magnetic north.** A phone's magnetometer reads magnetic north. The difference (magnetic declination) exceeds 20 degrees in parts of the world, and ignoring it points the user visibly off the Kaaba. Android's `GeomagneticField` is the platform's own World Magnetic Model implementation and supplies the correction, so no library or lookup table is needed.

Bearings are verified against the Aladhan Qibla API across ten cities on five continents, agreeing to within a hundredth of a degree. See `QiblaEngineTest`.

Heading comes from `TYPE_ROTATION_VECTOR`, the platform's own sensor fusion, which is far steadier than fusing the accelerometer and magnetometer by hand. That pair is kept only as a fallback for devices without a rotation vector. Readings pass through a circular low-pass filter, smoothed on the unit vector rather than on degrees so the needle does not swing wildly across the 359 to 0 boundary.

The screen reports sensor accuracy and prompts for a figure-of-eight recalibration when it drops. When there is no compass at all, the app still states the bearing from true north so a user with a separate compass can act on it.

Arriving is a state with hysteresis, not a comparison. Facing the Kaaba is announced with a vibration, so it must fire when the user arrives and at no other time. A single threshold cannot do that: a real compass at rest still drifts a degree or two, and a reading resting on the threshold crosses it repeatedly. `QiblaEngine.staysAligned` therefore takes the previous answer as an input — you arrive within 5 degrees and keep having arrived until you are 8 away — and the screens additionally refuse to repeat the pulse inside five seconds, and never fire it for the first heading of a session. This is measured behaviour, not caution: with a plain 5-degree test the app buzzed 26 times in 76 seconds on a phone lying still on a desk. `docs/HANDOVER.md` §5.9 carries the numbers, and `QiblaEngineTest` fails if the two angles are ever made equal.

3.10 Ramadan and Eid (1.4.0)

`HijriCalendar` in `:core` is the only file that touches `HijrahDate`. It gives the Hijri day for a date and the user's date shift (plus or minus 2 days), whether it is Ramadan, a `RamadanInfo` (Hijri year, first and last day, length, day number, and the two visibility windows below) and the date of Eid al-Fitr (1 Shawwal) and Eid al-Adha (10 Dhul Hijjah). There are no yearly tables anywhere, so 2027 and later need no change. Ramadan 1448 is 8 Feb to 8 Mar 2027 (29 days).

- **Suhoor ends at Fajr and iftar is at Maghrib**, exactly the engine's two times for the day, including the user's per prayer adjustments. No imsak margin and no precaution minutes.
- **Visibility is a function of the user's own calendar.** An eve notice from 7 days before day 1 through day 2, a timetable (and PDF) offered from 30 days before day 1 to the last day, the card only inside Ramadan, all of it gone at local midnight into 1 Shawwal.
- **No country table.** The app does not decide when Ramadan starts for anyone: it applies Umm al-Qura and the user's shift, and says so.
- **Port to iOS as it is.** `Calendar(identifier: .islamicUmmAlQura)` is the same calendar. The windows and the Eid rule are pure date arithmetic.

---

4. Notifications and reliability

Prayer alerts are the feature most likely to fail silently, so the schedule is rebuilt from five independent triggers:

1. every time the app is opened
2. every time an alarm fires (chains the next window)
3. a `WorkManager` job every 12 hours
4. device boot, time-zone change, clock change, and app update
5. the exact-alarm permission being granted or revoked (Android 12+)

Alarms are laid down two days ahead, so a single missed trigger cannot break the chain.

Exact alarm strategy

> **Corrected 1 Aug 2026.** This table used to say `setAlarmClock` was "permission-free" and was what the app fell back to when the permission was refused. **Both halves were wrong**, and it is the same misconception that caused a real `SecurityException` crash earlier in this project's history. `setAlarmClock` requires `SCHEDULE_EXACT_ALARM` like every other exact API — more strictly, in fact: its reference note is the only one of the three without an "unless the app is exempt from battery restrictions" clause.

Three rungs, each attempted only if the one above it failed, and every alert takes the highest rung available regardless of which style it will eventually make:

Rung	Mechanism	Guarantee
1	`setAlarmClock`	The only API Google documents as never moved: *"the system never adjusts their delivery time."* Needs `SCHEDULE_EXACT_ALARM`. Shows a status bar alarm icon.
2	`setExactAndAllowWhileIdle`	Exact but rate-limited, and the OS *"may take even more liberties"* when idle. Reachable without the permission if the user has exempted the app from battery optimisation.
3	`setAndAllowWhileIdle`	Inexact. Measured delivery window: **one hour**. Never throws.

`runCatching` at each rung rather than a permission check alone: some OEM builds and work profiles revoke the capability between the check and the call.

A late Fajr notification is worse than a status bar icon, which is why rung 1 is used even for quiet notifications. Onboarding asks for the permission on its last step, and both the home and Settings banners link to the system screen while it is missing.

Not solved by any of this — except it turned out to be. App Standby buckets are documented to hold a Restricted-bucket app to one alarm per day, and Google's public pages give `setAlarmClock` no exemption. The AOSP implementation does: `AlarmManagerService.isExemptFromAppStandby` short-circuits on `a.alarmClock != null`, so rung 1 never reaches the bucket branch. Confirmed on a device at bucket 45 (RESTRICTED) with the charger exemption off. What buckets still throttle is the app's *jobs* — the daily reschedule worker has no such exemption. An iOS port has no equivalent of either problem; an Android reimplementation has both. HANDOVER §6.

Alert style — chosen per prayer

Three answers, for each of the five prayers independently. A prayer with no style is silent; there is no separate on/off flag.

Style	Behaviour	Default
Off	No alert.	—
Notification	Heads-up notification with vibration. Respects Do Not Disturb.	**All five**
Alarm	Alarm-category alert with a sound the user picks, and it may sound through Do Not Disturb.	No

Per prayer rather than once for the app, because the request that drove it — *a loud alarm for Fajr, a quiet notification for the rest* — cannot be expressed by a single global choice.

Five full alarms a day, unasked for, is the sort of thing that gets an app uninstalled, so alarm mode is opt-in. When the user chooses it, they pick their own tone through the system ringtone picker, which already lists every alarm, ringtone and audio file on the device. No adhan recording is bundled: it would raise licensing questions and inflate the download of a charity app for a sound many users already have.

An alarm stays quiet while the phone is silenced, unless the user turns that off. Android deliberately does not mute the alarm stream with the ringer, which is right for a one-off wake-up alarm and wrong for a recurring alert; the notification still arrives on time and only the sound is dropped. Vibrate counts as silenced.

Sound is bound to the channel, not to the notification, and a channel's sound cannot be changed after creation. Choosing a new tone therefore creates a new channel id and deletes the old one, so the system Settings list does not fill with dead entries.

Do Not Disturb bypass needs notification policy access, which the user grants in system settings. Settings prompts for it only once alarm mode is chosen.

Channels

- `prayer_times` — IMPORTANCE_HIGH, vibrates. One notification per prayer.
- `prayer_alarm_v<hash>` — IMPORTANCE_HIGH, user's sound, bypasses Do Not Disturb.
- `next_prayer_badge` — IMPORTANCE_LOW, silent, ongoing. On by default, and free.

The badge refreshes when something has already woken the app rather than ticking every minute; a live-updating countdown would require a foreground service and a permanent battery cost for information already visible in the app.

Suhoor reminder and home screen widgets (1.4.0)

Two more alarm families share the manager, kept apart by action and request code because `PendingIntent` identity ignores extras. Prayers use codes up to 3,669 and `ACTION_PRAYER`. The optional suhoor reminder (Ramadan days only, 15 to 60 minutes before Fajr) uses

`ACTION_SUHOOR_REMINDER` and codes 10,001 to 10,366 and the same three rung ladder. The widgets use one non waking `RTC` alarm, code 20,000, set for the earlier of the next prayer and local midnight plus 2 seconds (exact when the phone allows exact alarms, inexact otherwise), and every redraw lays down the next one. A widget is a display, so it never uses `setAlarmClock` or a waking alarm and the `updatePeriodMillis` is 0. The widgets are `RemoteViews`, with no Glance, and read no location: they draw from the saved coordinates and settings. `docs/HANDOVER.md` §5.20 and §6 hold the detail.

---

5. Design system

Palette

Deep green (traditionally associated with Islam) as the primary, one amber accent reserved for warnings, warm-neutral paper surfaces in light and near-neutral ones in dark. Every hue carries meaning; nothing is decorative.

Role	Light	Dark
Primary	`#0E6B4F`	`#4FC48F`
Accent (warnings)	`#9A6208`	`#E8B14A`
Background	`#F7F5F3`	`#101312`
Surface	`#FDFCFB`	`#171B1A`
On surface	`#1B211E`	`#E7EAE8`
Secondary text	`#5A605C`	`#A9B2AE`

The full token set, and the reasoning behind each choice, is in `DESIGN_SYSTEM.md`. An iOS port should reproduce that document, not this table — this is a summary.

Two structural rules an implementation has to carry, not just the hex values:

- **Surfaces separate themselves by shadow in light and by a hairline border in dark.** A shadow on a dark surface is invisible; a border in light is a hard line the design does not want.
- **The next-prayer card is a fixed mint-to-sand gradient in light and flat in dark.** It must never be driven by the time of day: a background that moves through the day gives the countdown a different contrast ratio at Isha than at Fajr, and there is then no fixed pair left to assert.

Choosing a theme

`ThemeChoice` — `SYSTEM` (default), `LIGHT`, `DARK` — is persisted with the rest of the user's settings. `SYSTEM` resolves to light wherever the platform expresses no preference, so light is the effective default. The watch does not follow it; a wrist display is glanced at, and white on black is right on an OLED panel whatever the phone is set to.

Accessibility is enforced, not asserted. `ColorContrastTest` computes the WCAG 2.1 relative luminance of every foreground/background pair the UI actually uses and fails the build below 4.5:1 for text and 3:1 for icons and outlines. Editing a colour below threshold breaks the build.

Typography

The system font family, at system-scalable `sp` sizes, with nothing below 14sp. Bundling a webfont would add roughly 400KB to a charity app for no legibility gain, and would override the font a user may have chosen for accessibility reasons.

Time and countdown text uses tabular figures (`tnum`) so digits do not shift width as they change.

Other accessibility measures

- The next prayer is marked with a text label as well as a colour highlight, so the information does not depend on colour alone.
- The per-second countdown is hidden from screen readers and replaced with a readable summary ("Maghrib in 2h 14m"); announcing the digits every second would make the screen unusable with TalkBack.
- The Arabic bismillah carries an explicit content description.
- Touch targets are at least 48dp.
- Strings resolve through `stringResource`, so a language change while a screen is open re-reads correctly.

5a. Wear OS

The watch app is **standalone**. It calculates prayer times and the Qibla itself using `:core`, so it keeps working when the phone is out of range, switched off, or left at home.

- **App**: two swipeable pages, the prayer list with a live countdown, and a Qibla compass.
- **Tile**: next prayer, its time, and how long is left. Tiles are static snapshots, so rather than update every second the tile asks to be refreshed a minute after the next prayer begins. It is correct whenever the user looks at it and costs nothing in between.
- **Settings sync**: the phone publishes sect, madhab, method and location over the Data Layer. The transfer is one way and travels between two devices the same person owns. Nothing reaches a server. If no watch is paired, nothing is ever published.
- **Location**: the watch reads its own, either from onboard GPS or a fix the system hands over from the phone. A synced location covers watches without their own.

`minSdk` is 30 (Wear OS 3). Older watches run a different app model entirely and are not worth the maintenance for a solo project.

Trade-off worth knowing: the Data Layer is part of Google Play Services, so `play-services-wearable` is now a dependency of the phone app too. That is the only way to move data between a phone and a Wear OS watch. It sends nothing to any server, but it does mean the phone build is no

longer free of Google libraries. A no-GMS build flavour that simply drops watch sync is straightforward to add if that matters later.

5b. City lookup — a caution for the iOS port

Typed city names are resolved by the platform geocoder first (`android.location.Geocoder`), falling back to Open-Meteo's free geocoding API only when the phone has no geocoder backend.

This deliberately does **not** use Aladhan's `timingsByAddress`. That endpoint used to return the geocoded coordinates in its `meta` block and was the original implementation. It now returns a fixed placeholder — `8.8889, 7.7778` — for *every* address, so each search silently resolved to a point in Nigeria while the interface displayed the city the user had typed. Nothing failed, nothing logged, and the times looked plausible.

Two rules came out of that, and both should carry to any port:

- **Never display the text the user typed as if it were the resolved place.** Show the name that came back alongside the coordinates. Had the app done this, the fault would have read as "Lahore → Lagos, Nigeria" on the first try instead of going unnoticed.
- **A missing coordinate is a miss, not a default.** Parsing throws rather than falling back to zero, zero.

---

6. Privacy model

Data	Where it lives	Leaves the device?
Coordinates	DataStore, on device	Never
City name	DataStore, on device	Never
Sect, madhab, method	DataStore, on device	Never
Notification settings	DataStore, on device	Never
Ramadan card switch, dismissals, suhoor reminder choice (1.4.0)	DataStore, on device	Never
Widgets (1.4.0)	Drawn from the saved location and settings, no location read of their own	Never
Typed city name (fallback only)	Resolved on-device where possible, otherwise sent once to Open-Meteo	Only if the phone's own geocoder cannot answer, and with prior on-screen disclosure
Sect, madhab, method, location	Published to a paired watch	Only to the user's own watch, over the local Data Layer

Cloud backup **and device-to-device transfer** are both disabled. Android's backup service would otherwise copy the cached coordinates off the device, contradicting the app's own privacy promise. Re-entering settings takes two taps; the guarantee is worth more.

Two declarations are needed, not one. `allowBackup="false"` covers Android 11 and below. From Android 12 it stops cloud backup but leaves device-to-device transfer enabled on many devices, so both modules also set `android:dataExtractionRules`, excluding every domain from `<cloud-backup>` and `<device-transfer>`. An iOS port needs the same care: the equivalent is excluding the store from iCloud and iTunes backup.

Permissions requested: `ACCESS_COARSE_LOCATION`, `POST_NOTIFICATIONS`, `SCHEDULE_EXACT_ALARM`, `RECEIVE_BOOT_COMPLETED`, `INTERNET`. No fine location, no background location. 1.4.0 (Ramadan pack and widgets) adds none.

---

7. Testing

All unit tests are offline and deterministic.

Suite	Covers
`PrayerEngineTest`	Reference timetables for Makkah, London and Tehran; the Jafari Maghrib rule; madhab differences; high-latitude behaviour; the Ramadan adjustment; next-prayer roll-over and midnight spillover; chronological ordering across 3 cities × 4 methods × 52 weeks.
`QiblaEngineTest`	Bearings for ten cities on five continents, plus distance and normalisation — and the arrival hysteresis, including a test that fails if its two angles are ever collapsed into one.
`DeterminismTest`	Repeated computation stability, minute-boundary alignment, madhab equivalence.
`ColorContrastTest`	WCAG AA for every colour pair in both themes.
`CityLookupParseTest`	Geocoding response parsing: coordinates come from the response, the displayed name is the resolved place rather than the typed text, and a missing coordinate is a miss rather than zero, zero.
`TileFormatTest`	Watch tile countdown wording at the boundaries, and the refresh floor that stops a passed prayer re-rendering the tile in a loop.
`WearSettingsTest`	Settings synced from the phone reach the engine intact; an unpaired watch still calculates; the phone-to-watch key contract.
`PolarAndHemisphereTest`	Every 0.5° pole to pole × every day of 2026 × both madhabs is chronologically ordered; polar latitudes produce times rather than a crash; the reference latitude follows the method; **twelve rows of Moonsighting's own published timetable for Luleå, Trondheim and Helsinki are pinned as golden values** so the reverted 60° slide cannot be re-implemented silently.
`AdjustmentTest`	"Match your mosque": per-prayer corrections move a prayer by exactly that many minutes on every path, out-of-range values are clamped, and the worst legal pair still cannot invert a day.
`CoordinatesTest`	`Coordinates.orNull` rejects off-globe values, NaN, infinity and half-pairs — the gate on everything the watch's exported Data Layer listener is handed.
`LocaleDisciplineTest`	The app formats in its own language, not the device's; every translation declares its tag; **the shipped app is left-to-right and may not select a language it has no words in.**
`NoTranslationsYetTest`	Fails the build if any language-qualified `values-<lang>/` folder appears anywhere — the machine-translation ban in `CLAUDE.md`.
`DisclaimerContentTest`	The religious disclaimer still makes all four of its points, the dua request is still last and appears once, and the published copies have not drifted apart.
`BidiTest`	Right-to-left ordering asserted against `java.text.Bidi`, including a test that pins the *wrong* fix so it cannot be "simplified" back in.

Suite	Covers
`PlaceNameTest`	The town wins over the county; a point of interest never outranks an administrative area; a house number is never shown as a place.
`AlertStyleDowngradeTest`	An alarm rings, a projected time does not, an override brings it back, a silenced phone wins regardless — and a Notification is never upgraded to an Alarm.
`AlertCodecTest`	Per-prayer alert round-trip, rubbish dropped entry by entry, and the upgrade from the two keys it replaced.

18 suites, 145 tests as of 16 Aug 2026. `docs/HANDOVER.md` §9 carries the authoritative per-suite counts and the command that produces them; this table went three days stale listing only seven, which is why the count is stated here rather than left implicit. If you are reading this and the number disagrees with a run, trust the run — and fix the line.

Reference values were captured from the Aladhan API — an independent implementation of the same conventions — and pinned as golden values so the suite stays offline. To regenerate:

```
curl "https://api.aladhan.com/v1/timings/DD-MM-YYYY?latitude=..&longitude=..&method=N"
```

(method `0` = Shia Ithna-Ashari, `3` = MWL, `4` = Umm al-Qura)

---

8. Porting to iOS

Every calculation file in `core/` — `PrayerModels.kt`, `PrayerEngine.kt`, `QiblaEngine.kt`, `HijriCalendar.kt`, `WatchSyncContract.kt` — contains no Android imports, so everything in section 3 transfers directly.

The one exception is `PrayerLabels.kt`, which maps `PrayerSlot` and `CalcMethod` to translatable display names and therefore needs `android.content.Context`. It is kept in its own file precisely so the port can ignore it and supply a `Localizable.strings` lookup instead. The enums themselves carry no text.

1. Use adhan-swift, whose API mirrors adhan-java. Translate `PrayerEngine.kt` more or less line for line.
2. **Reimplement the Jafari Maghrib probe (3.4).** adhan-swift has the same missing angle.
3. **Set the high-latitude rule explicitly (3.5).** The Swift default has the same problem.
4. Carry over the Ramadan adjustment (3.6), minute truncation (3.7) and the Ramadan and Eid rules (3.10). Widgets are WidgetKit timelines on iOS, not alarms: the refresh strategy in `docs/HANDOVER.md` §5.20 does not carry over, the rule that they read no location does.
5. Replace `UserNotifications` for scheduling — iOS caps pending notifications at 64, so schedule roughly the next 12 days of prayers and top up on each app launch.
6. `UNUserNotificationCenter` cannot be woken on boot, so the "reschedule on boot" strategy does not apply; refresh on launch and on significant time change instead.

7. `PDFKit` replaces `PdfDocument`. `CoreLocation` with `kCLLocationAccuracyKilometer` replaces `LocationManager`.
8. Keep `ColorContrastTest` — port it, do not drop it.

8a. Disclaimer

The app states plainly, once after setup and again in Settings, that it is a helper and not a religious authority: that the author is not a Mufti or Aalim, that it was built with the help of artificial intelligence, that it may be wrong, and that anything doubtful should be checked with a local mosque or a qualified person.

It also has to establish four specific things — that the times are calculated on the device rather than supplied by any authority, how that calculation can fail, that a disagreement with the mosque is settled in the mosque's favour, and that the app carries no warranty — and it is published in four places that must be changed together. The full rule, the table of copies and the rejected alternatives are in `docs/HANDOVER.md` §5.15. A port to another platform inherits all of it.

---

9. Known limitations and next steps

Honest list of what is not covered:

- **High-latitude rule is not user-selectable.** `TWILIGHT_ANGLE` is a good default and matches Aladhan, but some UK mosques publish one-seventh-of-the-night times. ⚠️ This entry used to end "exposing this setting is the first thing to add". **That was struck from `docs/HANDOVER.md` on 2 Aug 2026 as simply false, and it is struck here for the same reason:** it is item A4 and it is *deliberately deferred*, because "Match your mosque" (below) closes the same gap without asking a user to understand twilight models. Do not build it on the strength of this bullet.
- **Per-prayer manual offsets — BUILT, 2 Aug 2026, as "Match your mosque".** Each prayer can be nudged by whole minutes and the Hijri date shifted by ±2 days; both sync to the watch. See `core/AdjustmentCodec.kt`, `SettingsRepository.setAdjustment`, and `AdjustmentTest` (11 tests). This bullet said "Not built" for three days after it shipped.
- **No bundled adhan audio.** Notification mode uses the system sound; alarm mode plays a tone the user picks from what is already on their phone. Shipping audio would add licensing questions and tens of megabytes for a file most users replace anyway.
- **City search needs a network** unless the phone's own geocoder can answer offline.
- **Times are shown in the phone's timezone, not the chosen city's.** This is right for the traveller the feature is built for, whose phone follows them, but someone checking a distant city from home sees those prayers on their own clock rather than the city's.
- **No instrumented UI tests.** Logic and colour are covered by unit tests; the Compose screens on both phone and watch are verified manually on emulators.

- ~~**App is unsigned.**~~ **Signed since 31 July 2026.** Both `:app` and `:wear` release bundles are signed from `keystore.properties`, which is gitignored and holds the owner's own upload key. ⚠️ **No agent may generate, hold, request or view that key** — see `CLAUDE.md` and `docs/RELEASING.md`. This bullet previously read "a release keystore must be generated before publishing", which invited exactly the hard-rule violation the project forbids.
- **No tablet-optimised layout.** The watch app and tile exist, and since 1.4.0 the phone has two home screen widgets (next prayer, and today's times).
- **The watch has *some* settings of its own.** Sect and madhab can be changed on the wrist — there is a school picker at the bottom of the times list, precisely so an unpaired watch is not stuck on Sunni/Shafi'i with Asr over an hour out. **Calculation method is still phone-only.** A watch that is never paired calculates with its own defaults and says which school produced the times.

---

10. Regenerating this document

The PDF beside this file is generated, never hand-made. After editing the markdown:

```
./tools/build-architecture-pdf.sh
```

The first release shipped a PDF a full version behind the markdown because it was produced by hand once and never again. Run the script rather than trusting the file on disk.

Made with love, free for the Ummah.